

Lab Assignment 6

Object-Oriented Programming

Branch

B.E., 2nd Year – ENC



Submitted By:
MRIDUL SHARMA

Roll No. 1024150204

Batch: 2023

Q1 - Implement a class Library that contains an array of Book objects. The Book class should have attributes like *title*, *author*, and *ISBN*. The Library class should provide the following methods:

(a) bool addNewBook(title, author, ISBN)

- Add a new book to the list
- Return type should be Boolean (true/false)
- Use pass-by-reference for arguments
- The function arguments and class attributes should have the same names •

Use the scope resolution operator to access class data members (b) bool

removeBooks(ISBN)

- Remove a book using ISBN
- Return Boolean (true/false)
- Use pass-by-reference
- Define this function outside the class using scope resolution operator

(c) displayDetails()

- Display all books in the library

Write a main function to:

- Add at least 5 books
- Remove 1 book
- Display all books **Code:**

WRITTEN CODE

```
#include <iostream>

using namespace

std; class Book {

public:

    string title, author, ISBN;

}; class

Library {

    Book books[10];
    int count;
public:
```

```

Library() { count = 0; } bool addNewBook(string &title, string
&author, string &ISBN) { if (count >= 10) return false;
books[count].title = title; books[count].author = author;
books[count].ISBN = ISBN; count++; return true;

}

bool removeBooks(string &ISBN);

void displayDetails() { for (int i =
0; i < count; i++) { cout <<
books[i].title << " " <<
books[i].author << " "
<< books[i].ISBN << endl;
}
}
};

```

```

bool Library::removeBooks(string &ISBN) {
for (int i = 0; i < count; i++) { if
(books[i].ISBN == ISBN) { for
(int j = i; j < count - 1; j++) {
books[j] = books[j + 1];
} count--;
return true;
}
}
return false;
} int main()
{ Library
lib; string t,
a, i;

t="A"; a="AA"; i="1"; lib.addNewBook(t,a,i);

t="B"; a="BB"; i="2"; lib.addNewBook(t,a,i);

```

```
t="C"; a="CC"; i="3"; lib.addNewBook(t,a,i);  
t="D"; a="DD"; i="4"; lib.addNewBook(t,a,i);  
t="E"; a="EE"; i="5"; lib.addNewBook(t,a,i); string  
rem = "3"; lib.removeBooks(rem);  
lib.displayDetails();  
}
```

```

1  #include <iostream>
2  using namespace std;
3  class Book {
4  public:
5      string title, author, ISBN;
6  };
7  class Library {
8      Book books[10];
9      int count;
10 public:
11     Library() { count = 0; }
12     bool addNewBook(string &title, string &author, string &ISBN) {
13         if (count >= 10) return false;
14         books[count].title = title;
15         books[count].author = author;
16         books[count].ISBN = ISBN;
17         count++;
18         return true;
19     }
20     bool removeBooks(string &ISBN);
21     void displayDetails() {
22         for (int i = 0; i < count; i++) {
23             cout << books[i].title << " "
24                  << books[i].author << " "
25                  << books[i].ISBN << endl;
26         }
27     }

```

```

28     };
29
30     bool Library::removeBooks(string &ISBN) {
31         for (int i = 0; i < count; i++) {
32             if (books[i].ISBN == ISBN) {
33                 for (int j = i; j < count - 1; j++) {
34                     books[j] = books[j + 1];
35                 }
36                 count--;
37                 return true;
38             }
39         }
40         return false;
41     }
42     int main() {
43         Library lib;
44         string t, a, i;
45         t="A"; a="AA"; i="1"; lib.addNewBook(t,a,i);
46         t="B"; a="BB"; i="2"; lib.addNewBook(t,a,i);
47         t="C"; a="CC"; i="3"; lib.addNewBook(t,a,i);
48         t="D"; a="DD"; i="4"; lib.addNewBook(t,a,i);
49         t="E"; a="EE"; i="5"; lib.addNewBook(t,a,i);
50         string rem = "3";
51         lib.removeBooks(rem);
52         lib.displayDetails();
53     }

```

```

A AA 1
B BB 2
D DD 4
E EE 5

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```

Q2 - Implement Question 1 using constructors with the following modifications: (a) Add:

- **Default constructor**
- **Parameterized constructor**
- **Copy constructor to initialize book details (*title, author, ISBN*)**

(b) Use the this pointer instead of scope resolution operator to access class data

members (c) Create function bool removeBooks(ISBN) same as Question 1

(d) Create function displayDetails() same as Question 1

(e) Create array of objects using:

- **Initializer list**
- **Dynamic initialization**

WRITTEN CODE

```
#include <iostream>

using namespace std;

class Book { public:
    string title, author, ISBN;

    // default
    Book() {}

    // parameterized
    Book(string title, string author, string ISBN)
    { this->title = title; this->author = author;
    this->ISBN = ISBN;

    }

    // copy
    Book(const Book &b) {
        this->title = b.title; this-
        >author = b.author; this->ISBN
        = b.ISBN;

    } void display() { cout << title << " " << author << " "
    << ISBN << endl; }
```

```
}; int

main() {

// initializer list
Book arr[2] = {

Book("A", "AA", "1"),
Book("B", "BB", "2") };

for (int i = 0; i < 2; i++)

arr[i].display();

// dynamic array

Book *b = new Book[2]{
Book("C", "CC", "3"),
Book("D", "DD", "4") };

for (int i = 0; i < 2;
i++) b[i].display();

delete[] b;

}
```

```

1 #include <iostream>
2 using namespace std;
3 class Book {
4 public:
5     string title, author, ISBN;
6     // default
7     Book() {}
8     // parameterized
9     Book(string title, string author, string ISBN) {
10         this->title = title;
11         this->author = author;
12         this->ISBN = ISBN;
13     }
14     // copy
15     Book(const Book &b) {
16         this->title = b.title;
17         this->author = b.author;
18         this->ISBN = b.ISBN;
19     }
20     void display() {
21         cout << title << " " << author << " " << ISBN << endl;
22     }
23 };
24 int main() {
25     // initializer list
26     Book arr[2] = {
27         Book("A", "AA", "1"),
28         Book("B", "BB", "2")
29     };
30     for (int i = 0; i < 2; i++)
31         arr[i].display();
32     // dynamic array
33     Book *b = new Book[2]{
34         Book("C", "CC", "3"),
35         Book("D", "DD", "4")
36     };
37     for (int i = 0; i < 2; i++)
38         b[i].display();
39     delete[] b;
40 }

```

```

A AA 1
B BB 2
C CC 3
D DD 4

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```

Q3 - Create a class Account with the following attributes:

- account number (*const long type*)
- transaction ID (*long type*)
- transaction type (*credit/debit — string*)
- balance (*double type*)

The class should provide the following methods:

(a) long depositAmount(to, from, amount)

- “to” and “from” are account numbers
- Return the transaction ID

(b) long creditAmount(to, from, amount)

- Similar to depositAmount
- Return the transaction ID

(c) void displayDetails()

- Display account number, balance, and transaction history •
- Make this a const function**

(d) In all functions, make all arguments const

Write a main function to:

- Create at least 5 Account objects
- Demonstrate credit and debit operations

WRITTEN CODE

```
#include <iostream> using
namespace std; class
Account { const long
account_number; long
transaction_id; string
transaction_type; double
balance; public:
Account(long acc, double bal) : account_number(acc) {
balance = bal; transaction_id = 0;
}
long depositAmount(const long to, const long from, const double amount) {
balance += amount;
transaction_type = "credit"; return
++transaction_id;
} long creditAmount(const long to, const long from, const double
amount) { balance -= amount; transaction_type = "debit"; return
++transaction_id;
} void displayDetails() const { cout
<< "Acc: " << account_number <<
" Balance: " << balance
```

```
<< " Last: " << transaction_type << endl;
} }; int
main() {
Account a1(1,1000), a2(2,2000), a3(3,3000),
a4(4,4000), a5(5,5000);
a1.depositAmount(1,2,500);
a2.creditAmount(2,1,300);
a1.displayDetails(); a2.displayDetails();
a3.displayDetails(); a4.displayDetails();
a5.displayDetails();
}
```

```

1 #include <iostream>
2 using namespace std;
3 class Account {
4     const long account_number;
5     long transaction_id;
6     string transaction_type;
7     double balance;
8 public:
9     Account(long acc, double bal) : account_number(acc) {
10         balance = bal;
11         transaction_id = 0;
12     }
13     long depositAmount(const long to, const long from, const double amount) {
14         balance += amount;
15         transaction_type = "credit";
16         return ++transaction_id;
17     }
18     long creditAmount(const long to, const long from, const double amount) {
19         balance -= amount;
20         transaction_type = "debit";
21         return ++transaction_id;
22     }
23     void displayDetails() const {
24         cout << "Acc: " << account_number
25              << " Balance: " << balance
26              << " Last: " << transaction_type << endl;
27     }
28 };
29 int main() {
30     Account a1(1,1000), a2(2,2000), a3(3,3000),
31            a4(4,4000), a5(5,5000);
32     a1.depositAmount(1,2,500);
33     a2.creditAmount(2,1,100);
34     a1.displayDetails();
35     a2.displayDetails();
36     a3.displayDetails();
37     a4.displayDetails();
38     a5.displayDetails();
39 }

```

```

Acc: 1 Balance: 1500 Last: credit
Acc: 2 Balance: 1700 Last: debit
Acc: 3 Balance: 3000 Last:
Acc: 4 Balance: 4000 Last:
Acc: 5 Balance: 5000 Last:

...Program finished with exit code 0
Press ENTER to exit console.

```

Q4 - Write a program to add data objects of two different classes using friend function.

WRITTEN CODE

```

#include <iostream>

using namespace std;

class B;

class A { int
x; public:

```

```

A() { x = 10; }
friend int sum(A, B);
}; class B { int y;
public: B() { y = 20;
} friend int sum(A,
B);
}; int sum(A a, B
b) { return a.x +
b.y; } int main()
{
A a; B b; cout << "Sum = " <<
sum(a, b); }

```

```

1 #include <iostream>
2 using namespace std;
3 class B;
4 class A {
5     int x;
6     public:
7         A() { x = 10; }
8         friend int sum(A, B);
9 };
10 class B {
11     int y;
12     public:
13         B() { y = 20; }
14         friend int sum(A, B);
15 };
16 int sum(A a, B b) {
17     return a.x + b.y;
18 }
19 int main() {
20     A a;
21     B b;
22     cout << "Sum = " << sum(a, b);
23 }

```

```

Sum = 30
...Program finished with exit code 0
Press ENTER to exit console.

```

Q5

Define a class **Complex** with properties (*real and imaginary*) and implement:

- (a) Parameterized constructor and copy constructor to initialize object values
- (b) void display() to display the complex number
- (c) void sum(Complex, Complex)

- Add two complex numbers
- Implement this as a **friend function**

Code:

```
#include <iostream>

using namespace std;

class Complex { float
real, imag;

public:
    Complex(float r = 0, float i = 0)
    { real = r; imag = i;
    }

    Complex(const Complex &c) {
real = c.real;

    imag = c.imag;
} void display() { cout << real << " + "
<< imag << "i\n"; } friend Complex
sum(Complex, Complex);
};

Complex sum(Complex c1, Complex c2)
{ Complex temp; temp.real =
c1.real + c2.real; temp.imag =
c1.imag + c2.imag; return
temp;
} int
main() {
Complex
c1(2,3),
c2(4,5);
Complex
c3 =
sum(c1,
```

```
c2);  
c3.display(  
);  
}
```

```
1  #include <iostream>  
2  using namespace std;  
3  class Complex {  
4      float real, imag;  
5  public:  
6      Complex(float r = 0, float i = 0) {  
7          real = r;  
8          imag = i;  
9      }  
10     Complex(const Complex &c) {  
11         real = c.real;  
12         imag = c.imag;  
13     }  
14     void display() {  
15         cout << real << " + " << imag << "i\n";  
16     }  
17     friend Complex sum(Complex, Complex);  
18 };  
19 Complex sum(Complex c1, Complex c2) {  
20     Complex temp;  
21     temp.real = c1.real + c2.real;  
22     temp.imag = c1.imag + c2.imag;  
23     return temp;  
24 }  
25 int main() {  
26     Complex c1(2,3), c2(4,5);  
27     Complex c3 = sum(c1, c2);  
28     c3.display();  
29 }
```

6 + 8i

...Program finished with exit code 0
Press ENTER to exit console.